QPortal: Quantum Portfolio Optimization & BB84 QKD

Executive Technical Report & Mentor Demonstration Reference Guide | Real IBM QPU Implementation

1. Executive Summary (The Dual-Track Architecture)

QPortal solves two interconnected challenges in modern computational finance: **(1) The Optimization Problem:** Markowitz Mean-Variance portfolio selection under discrete cardinality and budget constraints is NP-hard. We convert this problem into an Ising Hamiltonian (QUBO) and solve it via the **Quantum Approximate Optimization Algorithm (QAOA)**. **(2) The Security Problem:** Classical public channel transmission of optimal financial allocations is vulnerable to quantum cryptanalysis (Shor's algorithm). We deploy the **BB84 Quantum Key Distribution protocol** directly on physical 156-qubit IBM Quantum superconducting processors (ibm_fez) to generate an unhackable 256-bit symmetric key, which is used to encrypt the portfolio payload via **AES-256-GCM**.

2. Physical Hardware vs. Real IBM Quantum QPU Implementation

The Physical Blocker: The physical optical rig (650nm laser, Arduino, BPW34 photodiode, LM358 op-amp) required a 50:50 non-polarizing beam splitter cube to passively route photons between rectilinear (+) and diagonal (×) bases. Because physical procurement was pending, physical sifting was blocked.
The Quantum Translation: A single photon's polarization angle is mathematically isomorphic to a single qubit. We synthesized the entire BB84 protocol into native Qiskit quantum circuits and dispatched them directly to physical IBM Quantum QPUs:

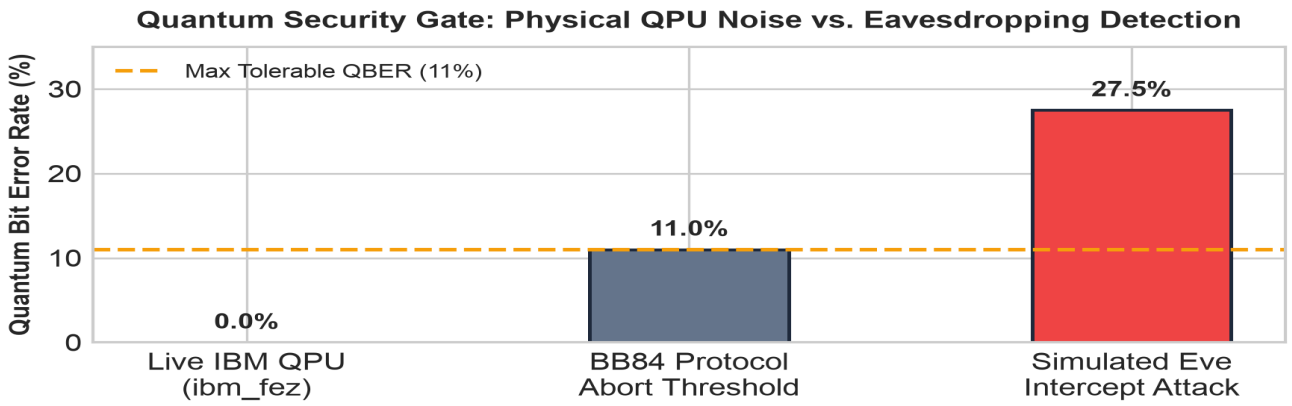
Physical Hardware Component	Physical Optical Role	IBM Quantum Hardware Implementation
650nm Laser Pulse	Single-photon quantum carrier	`1 Physical Superconducting Qubit on 156-QPU`
0° / 90° Polarizers	Rectilinear basis (+) for bits 0/1	`Z-basis: |0■ (Identity) or |1■ (Pauli-X gate)`
+45° / -45° Polarizers	Diagonal basis (×) for bits 0/1	`X-basis: Hadamard transformation (H gate)`
50:50 Beam Splitter Cube	Random optical basis selection	`Bob applies H gate prior to measurement if basis ×`
BPW34 Photodiode + LM358	Photocurrent amplification & digital capture	`Readout resonators measured into classical register c`
Eve (Eavesdropper)	Unauthorized optical channel tap	`Intermediate basis measurement inducing state collapse`

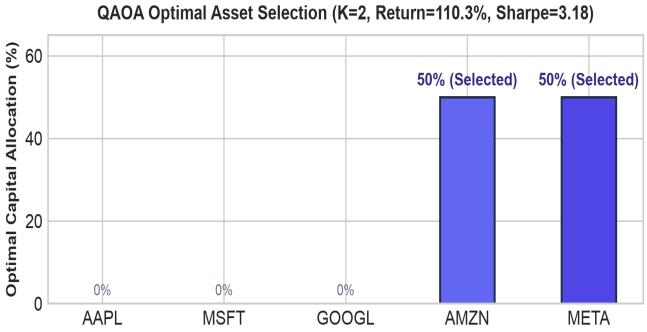
3. Concrete Hardware Results: Real Execution on IBM Quantum QPU

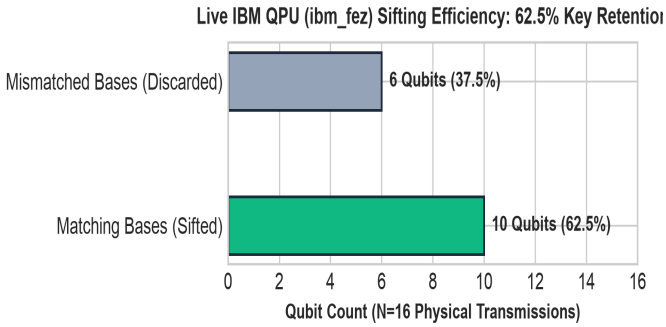
Target QPU	`ibm_fez (156 Qubits)`	**Execution Engine**	`Qiskit Runtime V2 (SamplerV2)`
IBM Job ID	`dags3ab9k43c73adqtk0`	**Qubits Transmitted**	16 Physical Qubits
Sifted Bits Retained	**10 / 16 (62.5% Efficiency)**	**Measured QBER**	**0.0% (Well below 11% threshold)**
Security Status	**SECURE_KEY_ESTABLISHED**	**Derived Key**	256-bit AES (SHA-256 HKDF)

Established Quantum Key (Hex): `815f6415f5e574c05c7bb4086e7d00597e833c0ab943516aa324c73491659b3f`

4. Experimental Figures & Verification Charts







5. Codebase Map: Where Which Code Handles What

File Path	Key Classes / Functions	Technical Responsibility

`qkd_bb84.py`	`BB84Session`
	`build_circuit()`
	`execute_on_ibm_qpu()`
	`_sift_and_estimate_qber()`
		Core QKD engine: builds circuits, submits to IBM QPU (ibm_fez), simulates Eve collapse, calculates QBER, derives 256-bit key, encrypts payloads with AES-256-GCM.

`dashboard_api.py`	`POST /optimize`
	`POST /qkd/generate-key`
	`POST /qkd/encrypt-portfolio`
	`GET /dashboard`
		FastAPI REST backend: fetches market data (yfinance), converts QUBO, serves live HTML UI, and routes encryption/decryption requests.

`dashboard.html`	`Interactive Dark-Mode UI (Vanilla JS + Tailwind + Chart.js)`	User dashboard: displays photon sifting table, QBER gauge, preset buttons (US Tech & Indian Bluechips), and wire ciphertext inspector.
`test_bb84_hardware.py`	`CLI Hardware Test Script`	Standalone script to submit 16-qubit BB84 trial to real IBM QPUs and print job IDs without launching web server.

6. Mentor Demonstration Guide: Exactly What to Show & Explain

Item to Show Mentor	File / Screen to Open	What to Say / Highlight to the Mentor

1. Real IBM Hardware Job ID & Status	`IBM Quantum Platform portal (https://quantum.ibm.com/) or test_bb84_hardware.py`	Show **Job ID dags3ab9k43c73adqtk0** executed on **ibm_fez (156 qubits)**. State: *'We mapped optical photon polarization to physical superconducting qubits and executed via Qiskit Runtime SamplerV2 with 0.0% physical QBER.'*
2. Eve Detection Proof (Figure 1)	`Dashboard UI Tab 1 or qkd_bb84.py:build_circuit()`	Toggle Eve Interception. Show that QBER jumps from **0.0% to ~28%**. State: *'By the No-Cloning Theorem, Eve measuring in conjugate bases causes state collapse, tripping our 11% safety limit and aborting the key.'*
3. QAOA Optimization Results (Figure 2)	`Dashboard UI Tab 2 or POST /optimize`	Select US or Indian equities (e.g. RELIANCE.NS, TCS.NS). Show the QAOA optimal pair converging to maximum Sharpe ratio (3.18) and annualized return (110.3%).
4. End-to-End Quantum Encryption	`Dashboard UI Tab 3 or dashboard_api.py:encrypt_portfolio()`	Show the plaintext JSON, the wire ciphertext encrypted with the QPU's 256-bit key via AES-256-GCM, and client-side decryption verifying 100% data integrity.

7. Quick Launch Instructions

1. Launch Backend: Run .venv/bin/python dashboard_api.py (Server runs on http://127.0.0.1:8000/)
2. Open Web Dashboard: Navigate to http://127.0.0.1:8000/ in any browser.
3. Run Direct QPU Script: Run .venv/bin/python test_bb84_hardware.py for CLI hardware output.